

---

## BCA EXAMINATION NOTES

Course Code: CA-122

# CA-122 Introduction to Operating System

---

### Exam Pattern:

Q1. Attempt Any Six – 6 marks

Q2. Attempt Any Three – 6 marks

Q3. Attempt Any Two – 6 marks

Q4A. Attempt Any One – 4 marks | Q4B. Compulsory – 2 marks

Q5. Attempt Any One – 6 marks

---

# UNIT 1: Introduction to Operating System

---

## 1.1 Introduction to OS

An Operating System (OS) is a system software that acts as an intermediary between the user and the computer hardware. It manages hardware resources and provides services for computer programs.

The OS is the most important system software; without it, hardware is useless. Examples: Windows, Linux, macOS, Android, iOS.

Primary Functions of OS: Process management, Memory management, File system management, Device management, User interface, Security and protection.

## 1.2 Components of Operating System

- Kernel: Core part of OS that manages hardware directly. Always loaded in memory.
- Shell: Interface between user and kernel. Accepts user commands.
- Process Manager: Creates, schedules, and terminates processes.
- Memory Manager: Allocates and deallocates memory to processes.
- File System Manager: Manages file creation, deletion, reading, writing.
- Device Manager: Controls hardware devices via device drivers.
- Network Manager: Manages network communication.
- Security Module: Controls access and protection.

## 1.3 Operating System Operations

The OS performs several critical operations:

- Dual-mode Operation: User mode (restricted) and Kernel mode (privileged). Prevents user programs from accessing hardware directly.
- Interrupt Handling: OS responds to hardware and software interrupts.
- System Call Interface: Programs request OS services through system calls.
- Booting: Loading the OS from disk to memory during system startup.
- Job Scheduling: Deciding which process gets CPU next.

## 1.4 Protection and Security

Protection: Mechanism to control access of processes and users to resources defined by the computer system. Ensures each program uses only what it should.

Security: Defense against external and internal attacks. Includes authentication (verifying identity), authorization (permission control), and encryption.

Techniques: User authentication (passwords, biometrics), access control lists, memory protection (base/limit registers), encryption.

## 1.5 Computing Environment

- Standalone Computing: Single computer used independently
- Client-Server Computing: Clients request services from centralized servers
- Distributed Computing: Multiple computers networked to share tasks
- Cloud Computing: Resources provided via internet (AWS, Azure, GCP)
- Mobile Computing: Computing on handheld devices (smartphones, tablets)
- Virtual Machine Environment: Multiple OS instances on single hardware

# UNIT 2: Overview of Operating System

---

## 2.1 Abstract View of OS

### User View

From user's perspective, the OS provides a convenient interface. For PC users: GUI (windows, icons, menus). For server users: resource utilization is key. For embedded/mobile: minimal interface, automated.

### System View

From system's perspective, OS is a resource allocator – manages CPU, memory, I/O devices. OS is a control program – controls program execution to prevent errors and misuse.

### Operating System Services

- Program Execution: Load and run programs
- I/O Operations: Manage input and output
- File System Manipulation: Create, delete, read, write files
- Communication: Between processes (IPC) or network
- Error Detection: Detect and respond to errors
- Resource Allocation: Allocate CPU, memory to multiple users
- Accounting: Track resource usage
- Protection/Security: Control access

## 2.2 System Calls

System calls provide the interface between a running program and the OS. Programs use system calls to request OS services.

Types of System Calls:

- Process Control: `fork()`, `exec()`, `exit()`, `wait()` – create, manage processes
- File Management: `open()`, `read()`, `write()`, `close()`, `delete()` – file operations
- Device Management: `request()`, `release()`, `read()`, `write()` – device operations
- Information Maintenance: `getpid()`, `alarm()`, `sleep()` – get/set system info
- Communication: `socket()`, `bind()`, `send()`, `receive()` – network/IPC
- Protection: `chmod()`, `umask()` – access control

## 2.3 Computer System Architecture

### Single-Processor Systems

Contains one main CPU capable of executing a general-purpose instruction set. May have special-purpose processors for specific devices (disk, keyboard). All user processes run on the one main CPU.

### Multiprocessor Systems

Also called parallel systems or tightly-coupled systems. Contain two or more processors sharing memory and buses.

Advantages: Increased throughput, economy of scale (shared resources), increased reliability (if one CPU fails, others continue).

Types: Symmetric Multiprocessing (SMP) – each CPU runs same OS; Asymmetric Multiprocessing – one master CPU and others are slaves.

## 2.4 Types of Operating Systems

- Batch OS: Jobs are collected into batches and processed without user interaction. CPU processes one job at a time. Example: IBM OS/360. Advantage: Efficient CPU use. Disadvantage: Long waiting time.
- Multi-programmed OS: Multiple programs loaded in memory simultaneously. When one job waits for I/O, CPU switches to another. Increases CPU utilization.
- Time-Shared OS (Multitasking): Multiple users interact simultaneously. CPU switches rapidly between processes (time slices). Example: Unix. Response time is small.
- Real-Time OS: Strict time constraints. Must respond within defined deadlines. Types: Hard RTOS (miss deadline = failure, e.g., pacemaker) and Soft RTOS (tolerate some delay, e.g., multimedia).
- Distributed OS: Runs on multiple processors connected by network. Resources are shared across nodes. Examples: Amoeba, CHORUS.

## 2.5 Process Management

Process: A program in execution. A process includes program code, program counter, registers, stack, and data section.

Process States: New → Ready → Running → Waiting → Terminated

Operations on Processes:

- Process Creation: `fork()` creates new process (child)
- Process Scheduling: Selecting which process runs on CPU (FCFS, SJF, Round Robin)
- Process Execution: Dispatching process to CPU
- Process Synchronization: Coordinating cooperating processes
- Process Termination: `exit()` ends a process; parent may use `wait()`
- Inter-process Communication (IPC): Shared memory, message passing

# UNIT 3: Basics of Linux

---

## 3.1 Introduction to Linux

Linux is a free, open-source, Unix-like operating system. Developed by Linus Torvalds in 1991. Based on Unix principles. Known for stability, security, and flexibility. Used in servers, mobile (Android), embedded systems.

## 3.2 Architecture of Linux

- Hardware: Physical devices (CPU, RAM, disk, keyboard)
- Kernel: Core of Linux – manages hardware, processes, memory, file systems
- Shell: Command interpreter – takes user commands and passes to kernel
- Utilities/Applications: User programs, commands, libraries

## 3.3 Basic Commands

**cal**: Displays calendar. `cal 2024` shows year calendar

**date**: Shows current date and time

**who**: Shows logged-in users

**whoami**: Shows current user name

**pwd**: Print Working Directory – shows current directory path

**ls**: Lists files and directories. `ls -l` for long format, `ls -a` for hidden files

**cat:** Displays file contents. `cat file.txt`  
**cp:** Copies file. `cp source dest`  
**mv:** Moves or renames file. `mv old new`  
**rm:** Removes file. `rm file.txt`  
**mkdir:** Creates directory. `mkdir dirname`  
**cd:** Changes directory. `cd /home`, `cd ..` goes up  
**rmdir:** Removes empty directory  
**chmod:** Changes file permissions. `chmod 755 file`  
**head:** Displays first 10 lines. `head -n 5 file`  
**tail:** Displays last 10 lines  
**sort:** Sorts file content alphabetically  
**grep:** Searches for pattern. `grep 'word' file.txt`

### 3.4 Types of Files in Linux

- Regular File (-): Normal text or binary files
- Directory File (d): Contains other files/directories
- Link File (l): Symbolic link pointing to another file
- Character Device (c): Character-based devices (keyboard, mouse)
- Block Device (b): Block-based devices (hard disk)
- Socket (s): For inter-process communication
- Named Pipe (p): FIFO – data channel between processes

### 3.5 chmod Command

`chmod` changes file permissions. Permissions: `r(read=4)`, `w(write=2)`, `x(execute=1)` for Owner, Group, Others.

Example: `chmod 755 file` – Owner: `rw(7)`, Group: `r-x(5)`, Others: `r-x(5)`

Symbolic mode: `chmod u+x file` (add execute to user), `chmod go-w file` (remove write from group/others)

### 3.6 Basic Filters

- head: `head -n 5 file` – shows first 5 lines
- tail: `tail -n 5 file` – shows last 5 lines
- sort: `sort file.txt` – sorts alphabetically; `sort -n` for numeric
- grep: `grep 'pattern' file` – searches for pattern; `-i` for case-insensitive; `-v` for non-matching lines

### 3.7 VI Editor

`vi` is a powerful text editor in Linux with three modes:

- Command Mode (default): Navigate, delete, copy, paste. Press `Esc` to enter.
- Insert Mode: Type/edit text. Press `i`, `a`, or `o` to enter from command mode.
- Ex Mode (Last Line): Save, quit, search. Press `:` to enter from command mode.

Important `vi` commands: `i` – insert before cursor | `a` – append after cursor | `:w` – save | `:q` – quit | `:wq` – save and quit | `:q!` – quit without saving | `dd` – delete line | `yy` – copy line | `p` – paste | `/word` – search

## UNIT 4: Shell Programming

---

### 4.1 Shell Environment and Scripting

Shell is a command interpreter that provides interface between user and kernel. A shell script is a text file containing a sequence of shell commands executed as a program.

Types of Shells: Bourne Shell (sh), Bash (Bourne Again Shell), C Shell (csh), Korn Shell (ksh), Z Shell (zsh)

Creating a script: Create file with .sh extension | First line: `#!/bin/bash` | Make executable: `chmod +x script.sh` | Run: `./script.sh` or `bash script.sh`

## 4.2 Conditional Statements

if Statement: `if [ condition ]; then commands; fi`

if-else: `if [ condition ]; then cmds; else cmds; fi`

if-elif-else: `if [ cond1 ]; then cmds; elif [ cond2 ]; then cmds; else cmds; fi`

case Statement: `case $var in val1) cmds ;; val2) cmds ;; *) default ;; esac`

## 4.3 Looping

for loop: `for var in list; do commands; done` — or — `for ((i=1; i<=5; i++)); do echo $i; done`

while loop: `while [ condition ]; do commands; done`

until loop: `until [ condition ]; do commands; done` — executes until condition becomes true

break: Exits the loop immediately.

continue: Skips current iteration and continues with next.

## 4.4 read and echo

echo: Prints output. `echo 'Hello World'` — `echo -n` (no newline) — `echo -e` (interpret escape sequences)

read: Reads input from user. `read var` — `read -p 'Enter name: ' name` — `read -s` (silent, for passwords)

## DEFINITIONS (2-Mark Ready)

---

**Operating System:** An OS is system software that manages hardware resources and provides services to programs and users.

**Process:** A process is a program in execution, including its code, data, and state information managed by the OS.

**System Call:** A system call is a programmatic interface through which a program requests services from the operating system kernel.

**Shell:** A shell is a command-line interpreter that acts as an interface between the user and the OS kernel, processing user commands.

**Kernel:** The kernel is the core component of the OS that directly manages hardware resources including CPU, memory, and devices.

**Multitasking:** Multitasking is the ability of an OS to run multiple processes apparently simultaneously by rapidly switching the CPU between them.

**Real-Time OS:** A RTOS is an OS that guarantees process completion within specified time constraints (deadlines), used in time-critical applications.

**File:** A file is a named collection of related information stored on secondary storage managed by the OS file system.

**chmod:** chmod (change mode) is a Linux command used to change the read, write, and execute permissions of a file or directory.

**Shell Script:** A shell script is a text file containing a series of shell commands that can be executed as a program in Unix/Linux.

## 20 MOST IMPORTANT QUESTIONS WITH ANSWERS

---

### Q1. What is an Operating System? State its functions. [2M]

An Operating System (OS) is system software that acts as an intermediary between the user and computer hardware.

Functions:

1. Process management
2. Memory management
3. File system management
4. Device management
5. User interface (GUI/CLI)
6. Security and protection

### Q2. Explain components of Operating System. [4M]

1. Kernel: Core of OS; manages CPU, memory, devices directly. Always loaded in memory.
2. Shell: Interface between user and kernel. Accepts user commands.
3. Process Manager: Creates, schedules, terminates processes.
4. Memory Manager: Allocates/deallocates memory.
5. File System Manager: Manages files and directories.
6. Device Manager: Controls hardware via drivers.
7. Security Module: Manages access control and authentication.

### Q3. Explain types of operating systems with examples. [6M]

1. BATCH OS: Jobs collected and processed in batches without user interaction. E.g., IBM OS/360.
2. MULTI-PROGRAMMED OS: Multiple programs in memory; CPU switches when one waits for I/O.
3. TIME-SHARING OS: CPU switches rapidly between multiple users. Fast response time. E.g., Unix.
4. REAL-TIME OS: Hard deadlines – pacemaker, aircraft control. Soft deadlines – multimedia.
5. DISTRIBUTED OS: Multiple computers connected by network share resources. E.g., Amoeba.

Comparison:      Batch→throughput;      Time-sharing→response;      Real-time→deadlines;  
Distributed→resource sharing.

### Q4. What are system calls? Name its types with examples. [4M]

System calls are the interface between user programs and the OS. They allow programs to request services from the kernel.

Types:

1. Process Control: fork(), exec(), exit(), wait()
2. File Management: open(), read(), write(), close()
3. Device Management: ioctl(), read(), write()
4. Information: getpid(), gettimeofday()
5. Communication: socket(), send(), receive()
6. Protection: chmod(), umask()

When a system call is made, control transfers from user mode to kernel mode.

**Q5. Explain the architecture of Linux with diagram. [6M]**

Linux Architecture has four layers:

1. **HARDWARE LAYER:** Physical components – CPU, RAM, hard disk, keyboard, mouse, network card.

2. **KERNEL LAYER:** Core of Linux OS. Direct hardware control.

Components: Process scheduler, Memory manager, File system, Device drivers, Network stack.

Kernel types: Monolithic kernel in Linux.

3. **SHELL LAYER:** Command interpreter. Types: bash, sh, csh, ksh.

Takes user commands → interprets → passes to kernel → returns result.

4. **UTILITIES/APPLICATIONS:** User programs, system utilities, compilers, editors, web servers.

Communication: User → Shell → System Calls → Kernel → Hardware

Return path: Hardware → Kernel → Shell → User

**Q6. Explain basic Linux commands with examples. [6M]**

1. **ls:** Lists directory contents. `ls -l` (detailed), `ls -a` (hidden files)

2. **pwd:** Print Working Directory – shows current location

3. **cd:** Change Directory. `cd /home`, `cd ..`, `cd ~` (home)

4. **mkdir dirname:** Create new directory

5. **rmdir dirname:** Remove empty directory

6. **cat file.txt:** Display file contents

7. **cp source dest:** Copy file

8. **mv old new:** Move or rename file

9. **rm file:** Delete file

10. **who:** Show logged-in users

11. **date:** Show current date and time

12. **cal:** Show calendar

13. **grep 'word' file:** Search for pattern

14. **chmod 755 file:** Change permissions

**Q7. Explain file permissions and chmod command. [4M]**

Linux file permissions control who can read, write, or execute a file.

Three categories: Owner (u), Group (g), Others (o)

Three permissions: r=read(4), w=write(2), x=execute(1)

Viewing permissions: `ls -l` shows: `-rwxr-xr--`

First char: file type (- regular, d directory, l link)

Next 3: owner permissions (rwx)

Next 3: group permissions (r-x)

Last 3: others permissions (r--)

**chmod command:**

Numeric: `chmod 755 file` → owner: rwx(7), group: r-x(5), others: r-x(5)

`chmod 644 file` → owner: rw-(6), group: r--(4), others: r--(4)

Symbolic: `chmod u+x file` (add execute to owner)

`chmod go-w file` (remove write from group/others)



**Q8. Explain VI editor with its modes. [4M]**

VI is a powerful text editor in Linux with three modes:

**1. COMMAND MODE (Normal Mode):**

Default mode when VI opens.

Navigation: h(left), j(down), k(up), l(right)

Editing: dd(delete line), yy(copy/yank), p(paste)

Enter with Esc key.

**2. INSERT MODE:**

For typing/editing text.

Enter: i(before cursor), a(after cursor), o(new line below)

Exit: Esc key.

**3. EX MODE (Last Line Mode):**

For file operations.

Enter: : from command mode

Commands: :w (save), :q (quit), :wq (save+quit), :q! (force quit), :/word (search)

**Q9. Write a shell script to print numbers from 1 to 10. [4M]**

Shell Script using for loop:

```
#!/bin/bash
echo "Numbers from 1 to 10:"
for i in 1 2 3 4 5 6 7 8 9 10
do
echo $i
done
```

Using for with seq:

```
#!/bin/bash
for i in $(seq 1 10)
do
echo $i
done
```

Using while loop:

```
#!/bin/bash
i=1
while [ $i -le 10 ]
do
echo $i
i=$((i+1))
done
```

To run: chmod +x script.sh && ./script.sh

**Q10. Explain conditional statements in shell scripting. [6M]**

SHELL CONDITIONAL STATEMENTS:

**1. if Statement:**

```
#!/bin/bash
if [ $a -gt 0 ]
then
echo "Positive"
fi
```

```
2. if-else:
if [ $a -gt $b ]
then
echo "a is greater"
else
echo "b is greater"
fi
```

```
3. if-elif-else:
if [ $marks -ge 90 ]
then echo "A grade"
elif [ $marks -ge 75 ]
then echo "B grade"
elif [ $marks -ge 60 ]
then echo "C grade"
else echo "Fail"
fi
```

```
4. case Statement:
case $day in
Monday) echo "Start of week" ;;
Friday) echo "End of week" ;;
*) echo "Midweek" ;;
esac
```

Comparison operators: -eq, -ne, -lt, -le, -gt, -ge (numeric); =, != (string)

### **Q11. What are the types of files in Linux? [2M]**

Linux supports several file types identified by ls -l output:

1. Regular File (-): Ordinary files containing data, text, code
2. Directory (d): Container for other files
3. Symbolic Link (l): Shortcut pointing to another file
4. Character Device (c): Devices accessed char-by-char (keyboard, terminal)
5. Block Device (b): Block-accessed devices (hard disk)
6. Socket (s): Used for network/IPC communication
7. Named Pipe (p): FIFO used for IPC

### **Q12. Explain looping in shell scripting. [4M]**

LOOPS IN SHELL SCRIPTING:

```
1. FOR LOOP:
for var in list
do
```

commands

done

Example: for fruit in apple mango banana; do echo \$fruit; done

## 2. WHILE LOOP:

while [ condition ]

do

commands

done

Example: while [ \$i -le 5 ]; do echo \$i; i=\$((i+1)); done

## 3. UNTIL LOOP:

until [ condition ]

do

commands

done

Runs until condition becomes true (opposite of while)

## 4. BREAK and CONTINUE:

break: Exits loop immediately

continue: Skips rest of current iteration

### Q13. What is read and echo in shell scripting? [2M]

echo: Used to display output on terminal.

Syntax: echo 'message' or echo \$variable

Options: echo -n (no newline), echo -e (enable escape sequences like \n, \t)

Example: echo 'Hello World'; echo "Score: \$marks"

read: Used to read input from user.

Syntax: read variable

Options: read -p 'prompt' var (display prompt)

read -s var (silent input, for passwords)

Example: read -p 'Enter name: ' name; echo "Hello \$name"

### Q14. Explain the concept of protection and security in OS. [4M]

PROTECTION: A mechanism that controls access of processes and users to system resources.

Goal: Each process uses only what it should, preventing accidental or malicious misuse.

Methods:

1. Memory Protection: Base and limit registers
2. CPU Protection: Timer interrupts
3. I/O Protection: All I/O only through OS

SECURITY: Defense against external/internal attacks.

Threats: Viruses, malware, unauthorized access, denial of service.

Methods:

1. Authentication: Passwords, biometrics, multi-factor
2. Authorization: Access Control Lists (ACL)
3. Encryption: Protect data in transit and at rest
4. Firewall: Filter network traffic

5. Audit Logs: Track user activities

**Q15. What is process management? Explain process states. [4M]**

PROCESS: A program in execution with its own code, data, stack, and process control block (PCB).

Process States:

1. NEW: Process being created
2. READY: Process waiting to be assigned to CPU
3. RUNNING: Instructions being executed by CPU
4. WAITING/BLOCKED: Process waiting for I/O or event
5. TERMINATED: Process finished execution

State Transitions:

New → Ready (admitted)  
Ready → Running (scheduler dispatches)  
Running → Waiting (I/O request)  
Waiting → Ready (I/O complete)  
Running → Ready (interrupt/preempted)  
Running → Terminated (exit)

**Q16. Explain grep command with examples. [4M]**

grep (Global Regular Expression Print) searches for patterns in files.

Syntax: grep [options] 'pattern' filename

Examples:

grep 'hello' file.txt – search for 'hello'  
grep -i 'hello' file.txt – case-insensitive search  
grep -n 'hello' file.txt – show line numbers  
grep -v 'hello' file.txt – lines NOT containing 'hello'  
grep -c 'hello' file.txt – count matching lines  
grep -r 'hello' /dir – recursive search in directory  
grep -w 'hello' file.txt – match whole word only

With regular expressions:

grep '^a' file – lines starting with 'a'  
grep 'e\$' file – lines ending with 'e'  
grep 'a.c' file – any char between a and c

**Q17. What is dual-mode operation in OS? [2M]**

Dual-mode operation allows OS to protect itself and other system components.

Two modes:

1. USER MODE: Programs run with restricted privileges. Cannot directly access hardware. Mode bit = 1.
2. KERNEL MODE (Privileged Mode): OS runs with full access to hardware. Mode bit = 0.

When a system call is made: User mode → Kernel mode (trap) → OS executes service → returns to User mode.

Benefit: Prevents user programs from crashing the system or accessing others' data.

**Q18. Explain multiprocessor systems. [4M]**

Multiprocessor systems have two or more CPUs sharing memory and buses.

Types:

1. Symmetric Multiprocessing (SMP): All CPUs run same OS, equal capability. E.g., modern PCs.
2. Asymmetric Multiprocessing: One master CPU, others are slaves. Master assigns tasks.

Advantages:

1. Increased Throughput: More work done in same time
2. Economy of Scale: Shared resources (memory, peripherals)
3. Increased Reliability: If one CPU fails, others continue (graceful degradation)
4. Better for Parallel Programs

Challenges: Complex synchronization, harder OS design, cache coherence issues.

**Q19. What are basic filter commands in Linux? [4M]**

Filter commands process text files:

1. head: Shows first N lines (default 10)  
head -n 5 file.txt (first 5 lines)

2. tail: Shows last N lines (default 10)  
tail -n 5 file.txt (last 5 lines)  
tail -f file.txt (follow file, useful for logs)

3. sort: Sorts file content  
sort file.txt (alphabetical)  
sort -n file.txt (numeric)  
sort -r file.txt (reverse)  
sort -k2 file.txt (sort by 2nd field)

4. grep: Search for pattern (see Q16)

5. Filters can be combined using pipe (|):  
Example: cat file.txt | sort | grep 'a' | head -5

**Q20. What is computing environment? Explain different types. [2M]**

Computing environment refers to the context/setting in which computing tasks are performed.

Types:

1. Standalone: Single computer, no network
  2. Client-Server: Clients request services from centralized server
  3. Distributed: Tasks distributed across networked computers
  4. Cloud: Resources (CPU, storage) provided over internet (AWS, Azure)
  5. Mobile: Computing on smartphones and tablets
  6. Virtual Machine: Multiple OS instances on single hardware using hypervisor
- Modern environments often combine multiple types (e.g., mobile app using cloud servers).